

Курс: Углубленное машинное обучение и искусственный интеллект в Python

Практическое задание: Архитектура Transformer Encoder для классификации текстов

Цель работы: Изучить внутреннее устройство архитектуры Transformer, реализовать ключевые компоненты кодировщика (Encoder) с использованием функционального API TensorFlow/Keras, обучить модель на реальных текстовых данных и оценить её эффективность.

Шаг 1. Выбор датасета и токенизация (Hugging Face Hub)

Вам необходимо выбрать текстовый датасет для задачи классификации на платформе Hugging Face Datasets.

Рекомендуемые датасеты:

- rotten_tomatoes (бинарная классификация отзывов на фильмы)
- ag_news (мультиклассовая классификация новостных статей)

Порядок выполнения предобработки:

1. Загрузите выбранный датасет с помощью библиотеки datasets.
2. Инициализируйте готовый предобученный токенизатор от любой легковесной модели (например, distilbert-base-uncased).
3. Токенизируйте текстовые корпуса, установив фиксированное ограничение на максимальную длину последовательности (параметр max_len).
4. Преобразуйте подготовленные данные в формат конвейера tf.data.Dataset и настройте батчирование.

```
from datasets import load_dataset
from transformers import AutoTokenizer
import tensorflow as tf
```

```
# Загрузка данных и токенизатора
dataset = load_dataset("rotten_tomatoes")
tokenizer = AutoTokenizer.from_pretrained("distilbert-base-uncased")
```

Шаг 2. Реализация Positional Encoding (Встраивание позиций)

Поскольку механизм Attention обрабатывает все токены параллельно, он не учитывает порядок слов в предложении. Для сохранения пространственной структуры текста необходимо добавить позиционную информацию к семантическим эмбедингам.

Спецификация: Создайте кастомный слой TokenAndPositionEmbedding, наследуемый от `tf.keras.layers.Layer`. Слой должен принимать на вход индексы токенов, инициализировать внутри себя два независимых слоя Embedding (один для токенов, второй для их позиций) и возвращать поэлементную сумму полученных векторов.

Шаг 3. Реализация блока Transformer Encoder

Спроектируйте кастомный слой TransformerBlock, реализующий классическую архитектуру кодировщика Трансформера.

Архитектурные компоненты и требования:

1. **Multi-Head Attention:** Примените встроенный слой `layers.MultiHeadAttention`. Реализуйте механизм Self-Attention, передав входной тензор слоя в качестве векторов Query, Key и Value одновременно.
2. **First Add & Norm:** Организуйте сквозную связь (Skip-connection), добавив исходный входной сигнал к результату работы слоя Attention, и примените `layers.LayerNormalization`.
3. **Feed-Forward Network (FFN):** Постройте полносвязную подсеть из двух слоев Dense. Первый слой должен расширять промежуточную размерность (параметр `ff_dim`) и использовать функцию активации GELU. Второй слой возвращает размерность данных к базовому размеру эмбедингов (`embed_dim`).
4. **Second Add & Norm:** Повторно организуйте Skip-connection (сложение выхода FFN с результатом первой нормализации) и выполните финальную послойную нормализацию.
5. **Регуляризация:** Добавьте слои Dropout после механизма Attention и после сети FFN для предотвращения переобучения.

Шаг 4. Сборка и обучение финальной модели

Используя разработанные кастомные компоненты, соберите сквозную модель классификации через Keras Functional API:

1. **Входной слой:** `layers.Input(shape=(max_len,))`
2. **Слой эмбедингов:** Кастомный слой `TokenAndPositionEmbedding`.
3. **Обработка последовательности:** Как минимум 1 блок `TransformerBlock` (для усложнения структуры допускается последовательный стек из 2 блоков).
4. **Агрегация:** Слой `layers.GlobalAveragePooling1D()` для перехода от двумерной матрицы последовательности токенов к фиксированному вектору признаков.
5. **Выходной классификатор:** Слой `layers.Dense` с активацией `sigmoid` (для бинарного разделения) или `softmax` (для многоклассового режима).

Скомпилируйте нейросеть с оптимизатором Adam и обучите её в течение **3–5 эпох**.

Шаг 5. Оценка точности и инференс

1. Визуализируйте динамику обучения: постройте графики функций потерь (Loss) и метрики точности (Accuracy) отдельно для тренировочного и валидационного множеств.
2. Проведите финальное тестирование обученной модели на отложенной тестовой выборке. Рассчитайте метрики: **Accuracy**, **Precision**, **Recall** и **F1-score**.